

Lecture 4: MapReduce Programming & Execution

DSL351 / DSP352

Amit Kumar Dhar

ED1, 406A

amitkdhar@iitbhlai

January 12, 2026

Table of Contents

1. Introduction
2. Anatomy of a MapReduce Program
3. Hadoop Data Types (Writables)
4. Input and Output Formats
5. Case Study: Word Count
6. Compiling and Execution
7. Complete Example

Introduction

Recap: The MapReduce Paradigm

- **MapReduce** is a programming model for processing large data sets.
- **Map Phase:** Processes input key/value pairs to generate a set of intermediate key/value pairs.
- **Reduce Phase:** Merges all intermediate values associated with the same intermediate key.
- The framework handles the messy details:
 - Scheduling
 - Fault Tolerance
 - Data Distribution (Shuffle & Sort)

Today's Goals

- Understand the **Anatomy** of a MapReduce program in Java.
- Explore Hadoop **Data Types** (Writables).
- Learn about **InputFormats** and **OutputFormats**.
- **Deep Dive:** Implement the classic "Word Count" algorithm.
- Learn how to **Compile, Package, and Execute** the job on a cluster.

Anatomy of a MapReduce Program

The Three Main Components

A standard MapReduce application typically consists of three main classes (or a single class with inner classes):

1. **Mapper Class:** Contains the logic for the Map phase.
2. **Reducer Class:** Contains the logic for the Reduce phase.
3. **Driver Class:** The entry point (main method) that sets up the job configuration and triggers execution.

1. The Mapper Class

- Extends the generic 'Mapper' class.
- Generic Signature: Mapper<KEYIN, VALUEIN, KEYOUT, VALUEOUT>
- Overrides the map() method.

```
1 public class MyMapper
2     extends Mapper<LongWritable, Text, Text, IntWritable> {
3
4     @Override
5     public void map(LongWritable key, Text value, Context context)
6         throws IOException, InterruptedException {
7         // Map logic here
8     }
9 }
```

2. The Reducer Class

- Extends the generic 'Reducer' class.
- Generic Signature: `Reducer<KEYIN, VALUEIN, KEYOUT, VALUEOUT>`
- **Note:** 'KEYIN' and 'VALUEIN' must match the Mapper's 'KEYOUT' and 'VALUEOUT'.

```
1 public class MyReducer
2     extends Reducer<Text, IntWritable, Text, IntWritable> {
3
4     @Override
5     public void reduce(Text key, Iterable<IntWritable> values, Context context
6     )
7         throws IOException, InterruptedException {
8         // Reduce logic here
9     }
```

3. The Driver Class

- Configures the Job object.
- Sets Mapper, Reducer, Combiner classes.
- Sets Input/Output formats and paths.

```
1 public static void main(String[] args) throws Exception {  
2     Configuration conf = new Configuration();  
3     Job job = Job.getInstance(conf, "My Job Name");  
4     job.setJarByClass(MyDriver.class);  
5     job.setMapperClass(MyMapper.class);  
6     job.setReducerClass(MyReducer.class);  
7     // ... set output key/value classes ...  
8     System.exit(job.waitForCompletion(true) ? 0 : 1);  
9 }
```

The Context Object

- Both 'map()' and 'reduce()' take a Context object as an argument.
- It serves as the communication channel between your code and the Hadoop system.
- **Uses:**
 - Writing output: `context.write(key, value)`
 - Reporting progress (heartbeats).
 - Accessing job configuration parameters.
 - Managing counters.

Hadoop Data Types (Writables)

Why not Java Serialization?

- Java's native 'Serializable' is too heavyweight.
- It stores class names, version UIDs, and object overhead.
- **Big Data Requirements:**
 - **Compact:** Minimize network bandwidth and storage.
 - **Fast:** Fast serialization/deserialization.
 - **Comparable:** Keys must be sorted during the Shuffle phase.

The Writable Interface

Hadoop defines its own box classes that implement the `Writable` interface.

- **Writable:** Methods to serialize/deserialize fields.
 - `'write(DataOutput out)'`
 - `'readFields(DataInput in)'`
- **WritableComparable:** Subinterface for keys. Adds `'compareTo()'`.
 - Keys in MapReduce **must** implement `'WritableComparable'`.

Common Writable Types

Java Type	Hadoop Writable	Description
String	Text	UTF-8 formatted text
int	IntWritable	32-bit signed integer
long	LongWritable	64-bit signed integer
boolean	BooleanWritable	True/False
null	NullWritable	Placeholder (no value)
float	FloatWritable	32-bit float
double	DoubleWritable	64-bit double

Using Writables

- Writables are mutable (for efficiency).
- Do not create new objects inside loops if possible.

```
1 // Creating
2 IntWritable one = new IntWritable(1);
3 Text word = new Text();
4
5 // Setting value
6 word.set("Hello");
7
8 // Getting value (conversion to Java type)
9 int val = one.get();
10 String str = word.toString();
```

Input and Output Formats

The InputFormat class defines:

1. **Input Splits:** How input data is divided into splits (logical chunks). One Map task is created per split.
2. **RecordReader:** How to read data from a split and generate Key/Value pairs.

TextInputFormat (Default):

- **Key:** 'LongWritable' (Byte offset of the line).
- **Value:** 'Text' (Content of the line).

Other Input Formats

- `KeyValueTextInputFormat`: First token is key, rest is value.
- `SequenceFileInputFormat`: Efficient binary format for inter-job data.
- `NLineInputFormat`: Splits based on number of lines (N).
- `DBInputFormat`: Read from SQL databases.

The `OutputFormat` class defines:

1. **RecordWriter:** How to write Key/Value pairs to the output file.
2. **OutputCommitter:** Manages the lifecycle of task outputs (moving temp files to final location).

TextOutputFormat (Default):

- Writes "Key \t Value \n".

Case Study: Word Count

The "Hello World" of MapReduce

Objective: Count the frequency of every word in a large text dataset.

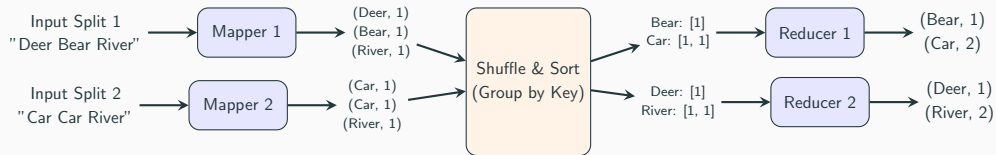
Input: Text files.

- Line 1: "Deer Bear River"
- Line 2: "Car Car River"
- Line 3: "Deer Car Bear"

Output: List of words and counts.

- Bear: 2, Car: 3, Deer: 2, River: 2

Logic Illustration



WordCount: The Mapper Code

```
1 public class TokenizerMapper
2     extends Mapper<Object, Text, Text, IntWritable>{
3
4     private final static IntWritable one = new IntWritable(1);
5     private Text word = new Text();
6
7     public void map(Object key, Text value, Context context)
8         throws IOException, InterruptedException {
9
10        StringTokenizer itr = new StringTokenizer(value.toString());
11        while (itr.hasMoreTokens()) {
12            word.set(itr.nextToken());
13            context.write(word, one);
14        }
15    }
16 }
```

- **Input Key:** 'Object' (ignored, usually LongWritable offset).
- **Input Value:** 'Text' (the line of text).
- **Operation:** Tokenizes the line into words.
- **Output:** Emits '(word, 1)' for every word found.
- **Optimization:** 'one' and 'word' are static/instance variables to reuse objects.

WordCount: The Reducer Code

```
1 public class IntSumReducer
2     extends Reducer<Text,IntWritable,Text,IntWritable> {
3
4     private IntWritable result = new IntWritable();
5
6     public void reduce(Text key, Iterable<IntWritable> values, Context context
7         )
8         throws IOException, InterruptedException {
9
10        int sum = 0;
11        for (IntWritable val : values) {
12            sum += val.get();
13        }
14        result.set(sum);
15        context.write(key, result);
16    }
```

Analysis of Reducer

- **Input:** Key (the word) and a List of Values (counts).
- **Example Input:** 'River', '[1, 1]'
- **Logic:** Iterate through the values and sum them up.
- **Output:** '(word, total_count)'.
- **Example Output:** 'River', '2'

WordCount: The Driver Code

```
1 public class WordCount {
2     public static void main(String[] args) throws Exception {
3         Configuration conf = new Configuration();
4         Job job = Job.getInstance(conf, "word count");
5         job.setJarByClass(WordCount.class);
6         job.setMapperClass(TokenizerMapper.class);
7         job.setCombinerClass(IntSumReducer.class); // Optional
8         job.setReducerClass(IntSumReducer.class);
9         job.setOutputKeyClass(Text.class);
10        job.setOutputValueClass(IntWritable.class);
11        FileInputFormat.addInputPath(job, new Path(args[0]));
12        FileOutputFormat.setOutputPath(job, new Path(args[1]));
13        System.exit(job.waitForCompletion(true) ? 0 : 1);
14    }
15 }
```

Combiner Class

- In the Driver code, we saw:
- `job.setCombinerClass(IntSumReducer.class);`
- **What is it?** A "Mini-Reducer" running locally on the Mapper node.
- **Purpose:** Reduces network traffic by pre-aggregating data before shuffle.
- **Condition:** Operation must be commutative and associative (like Sum, Max).

Compiling and Execution

Prerequisites

To run the code, you need:

1. **JDK 8 or higher** installed.
2. **Apache Hadoop** libraries (usually via Maven).
3. **Hadoop Cluster** (Single node or Multi-node) running.

1. Compilation (Maven)

Standard 'pom.xml' dependencies:

```
1 <dependencies>
2   <dependency>
3     <groupId>org.apache.hadoop</groupId>
4     <artifactId>hadoop-common</artifactId>
5     <version>3.3.1</version>
6   </dependency>
7   <dependency>
8     <groupId>org.apache.hadoop</groupId>
9     <artifactId>hadoop-mapreduce-client-core</artifactId>
10    <version>3.3.1</version>
11  </dependency>
12 </dependencies>
```

Command: mvn clean package

2. Data Preparation

Upload input files to HDFS:

```
1 # Create directory in HDFS
2 hdfs dfs -mkdir -p /user/amit/input
3
4 # Copy local file to HDFS
5 hdfs dfs -put local_file.txt /user/amit/input/
```

3. Running the Job

Use the `hadoop` command to submit the jar:

```
1 # Syntax: hadoop jar <jar_path> <main_class> <args>
2
3 hadoop jar wc.jar WordCount /user/amit/input /user/amit/output
```

Note: The output directory ('/user/amit/output') must **NOT** exist. Hadoop will create it. If it exists, the job fails (safety mechanism).

4. Inspecting Output

Check the success flag and read the output:

```
1 # List output directory
2 hdfs dfs -ls /user/amit/output
3 # Found 2 items
4 # -rw-r--r--    1 amit supergroup          0 2026-01-11 10:00 /user/amit/output
   /_SUCCESS
5 # -rw-r--r--    1 amit supergroup       25 2026-01-11 10:00 /user/amit/output
   /part-r-00000
6
7 # Cat the content
8 hdfs dfs -cat /user/amit/output/part-r-00000
```

Inspecting Execution: Web UI

Hadoop provides a Web Interface to monitor applications.

- **ResourceManager UI:** Typically at `http://localhost:8088`
- **NameNode UI:** Typically at `http://localhost:9870`

What to look for:

- Job State (ACCEPTED, RUNNING, FINISHED).
- Counters (Map input records, Reduce output records).
- Logs for individual tasks (stdout/stderr).

Complete Example

Summary: End-to-End Workflow

1. **Develop:** Write Mapper, Reducer, Driver in Java.
2. **Build:** Create a JAR file (e.g., 'wc.jar').
3. **Upload:** Put data onto HDFS.
4. **Run:** `hadoop jar wc.jar WordCount <in> <out>`
5. **Monitor:** Check port 8088.
6. **Retrieve:** 'hdfs dfs -cat' or '-get' the results.

Next Class: MapReduce Design Patterns - How to solve real problems beyond Word Count.

Questions?